HW2: Markov Models of Natural Language

Fill in your name and the names of any students who helped you below.

> I affirm that I personally wrote the text, code, and comments in this homework assignment.
>
> I received help from [other student] who gave me suggestions on [problem].

- [Aashna Sibal]

Comments are required for problems 1,2, and 3. They are not required for problem 4

Language Models

Many of you may have encountered the output of machine learning models which, when "seeded" with a small amount of text, produce a larger corpus of text which is expected to be similar or relevant to the seed text. For example, there's been a lot of buzz about the new GPT-3 model, related to its carbon footprint, bigoted tendencies, and, yes, impressive (and often humorous) ability to replicate human-like text in response to prompts.

We are not going to program a complicated deep learning model, but we will construct a much simpler language model that performs a similar task. Using tools like iteration and dictionaries, we will create a family of **Markov language models** for generating text. For the purposes of this assignment, an n-th order Markov model is a function that constructs a string of text one letter at a time, using only knowledge of the most recent n letters. You can think of it as a writer with a "memory" of n letters.

Data

Our training text for this exercise comes from the first 10 chapters of Jane Austen's novel *Emma*, which I retrieved from the archives at Project Gutenberg. Intuitively, we are going to write a program that "writes like Jane Austen," albeit in a very limited sense.

In [1]:
```
s = "CHAPTER I Emma Woodhouse, handsome, clever, and rich, with a comfortable home and happy disposition, seemed to
```

Comments and Docstrings

Here's what we expect:

- **Comments**: Use comments liberally to explain the purpose of short snippets of code.
- **Docstrings**: Functions (and, later, classes) should always be accompanied by a *docstring*. Briefly, the docstring should provide enough information that a user could correctly use your function ***without seeing the code*.** In somewhat more detail, the docstring should include the following information:
  - One or more sentences describing the overall purpose of the function.
  - An explanation of each of the inputs, including what they mean, their required data types, and any additional assumptions made about them.
  - An explanation of the output.

In future homeworks, as well as on exams, we will be looking for clear and informative comments and docstrings.

Code Structure

In general, there are many good ways to solve a given problem. However, just getting the right result isn't enough to guarantee that your code is of high quality. Check the logic of your solutions to make sure that:

- You aren't making any unnecessary steps, like creating variables you don't use.
- You are effectively making use of the tools in the course, especially control flow.
- Your code is readable. Each line is short (under 80 characters), and doesn't have long tangles of functions or `()` parentheses.

Ok, let's go!

Exercise 1

Write a function called `count_characters()` that counts the number of times each character appears in a user-supplied string `s`. Your function should loop over each element of the string, and sequentually update a `dict` whose keys are characters and whose values are the number of occurrences seen so far. You may know of other ways to achieve the same result. However, you should use the loop approach, since this will generalize to the next exercise.

Note: while the construct `for letter in s:` *will work for this exercise, it will not generalize to the next one. Use* `for i in range(len(s)):` *instead.*

Example usage:

```
count_characters("tortoise")
{'t' : 2, 'o' : 2, 'r' : 1, 'i' : 1, 's' : 1, 'e' : 1}
```

Hint: Yes, you did a problem very similar to this one on HW1.

Your Solution

In [2]:
```
# write count_characters() here

def count_characters(s):
    """
    Counts the number of times that each character appears in a user-supplied string s by looping over each element
    of the string and then updating a dict, where the keys are characters and the values are the number of
    occurences so far. Returns the dictionary.

    parameter 1: s, user-supplied string that the characters are counted from

    returns D, the updated dictionary with each character in the string as keys and the frequency as the values
    """
    D = {}                          # initializing an empty dictionary


    for i in (range(len(s))):       # accessing every index in the string for the string's whole length
        D[s[i]]=0                   # adding each unique character from s as a key in D

    for i in (range(len(s))):       # accessing every index in the string for the string's whole length
        D[s[i]]+=1                  # counting the number of times a character is in the string and assigning that
                                    # to its corresponding value in the dictionary

    return D                        # returning the updated dictionary


count_characters("tortoise")        # testing out the function
```

Out[2]:
```
{'t': 2, 'o': 2, 'r': 1, 'i': 1, 's': 1, 'e': 1}
```

Exercise 2

An `n`-*gram* is a sequence of `n` letters. For example, `bol` and `old` are the two 3-grams that occur in the string `bold`.

Write a function called `count_ngrams()` that counts the number of times each `n`-gram occurs in a string, with `n` specified by the user and with default value `n = 1`. You should be able to do this by making only a small modification to `count_characters()`.

Example usage:

```
count_ngrams("tortoise", n = 2)
```

```
{'to': 2, 'or': 1, 'rt': 1, 'oi': 1, 'is': 1, 'se': 1} # output
```

Your Solution

In [3]:

```
# write count_ngrams() here

def count_ngrams(s,n):
    """
    Counts the number of times each n-grams sequence occurs in a string. N is specified by the user and has the
    default value n = 1.
    parameter 1: s, user-supplied string that the n-gram occurances are being counted in
    parameter 2: n, specified by the user, sequence of letters to count, default value n=1

    returns D, the updated dictionary with the n-grams sequences as keys and the frequencies as the values
    """
    D = {}                          # initializing an empty dictionary

    for i in (range(len(s)-n+1)):   # indexing through the string only till the last set of "n" characters in it
                                    # so that no dictionary keys are less than n characters long
        D[s[i:i+n]]=0               # adding each grouping of characters from i up to, but not including, i+n
                                    # (or groupings that are n-characters long)
                                    # as keys in D

    for i in (range(len(s)-n+1)):   # indexing through the string only till the last set of "n" characters in it
        D[s[i:i+n]]+=1              # for every occurance of the specific characters of n-grams, adding 1 to the
                                    # corresponding value in D

    return D                        # returning the updated dictionary

count_ngrams("tortoise", n = 2)
```

Out[3]: `{'to': 2, 'or': 1, 'rt': 1, 'oi': 1, 'is': 1, 'se': 1}`

Exercise 3

Now we are going to use our `n`-grams to generate some fake text according to a Markov model. Here's how the Markov model of order `n` works:

A. Compute (`n`+1)-gram occurrence frequencies

You have already done this in Exercise 2!

B. Pick a starting (`n`+1)-gram

The starting (`n`+1)-gram can be selected at random, or the user can specify it.

C. Generate Text

Now we generate text one character at a time. To do so:

1. Look at the most recent `n` characters in our generated text. Say that `n = 3` and the 3 most recent character are `the`.
2. We then look at our list of `n+1`-grams, and focus on grams whose first `n` characters match. Examples matching `the` include `them`, `the`, `thei`, and so on.
3. We pick a random one of these `n+1`-grams, weighted according to its number of occurrences.
4. The final character of this new `n+1` gram is our next letter.

For example, if there are 3 occurrences of `them`, 4 occurrences of `the`, and 1 occurrences of `thei` in the n-gram dictionary, then our next character is `m` with probabiliy 3/8, `[space]` with probability 1/2, and `i` with probability `1/8`.

Remember: the ***3rd***-order model requires you to compute ***4***-grams.

What you should do

Write a function that generates synthetic text according to an `n`-th order Markov model. It should have the following arguments:

- `s`, the input string of real text.
- `n`, the order of the model.
- `length`, the size of the text to generate. Use a default value of 100.
- `seed`, the initial string that gets the Markov model started. I used `"Emma Woodhouse"` (the full name of the protagonist of the novel) as my `seed`, but any subset of `s` of length `n+1` or larger will work.

Demonstrate the output of your function for a couple different choices of the order `n`.

Expected Output

Here are a few examples of the output of this function. Because of randomness, your results won't look exactly like this, but they should be qualitatively similar.

```
markov_text(s, n = 2, length = 200, seed = "Emma Woodhouse")
```

```
   Emma Woodhouse ne goo thimser. John mile sawas amintrought will on I kink you kno but every sh
   inat he fing as sat buty aft from the it. She cousency ined, yount; ate nambery quirld diall
   yethery, yould hat earatte
```

```
markov_text(s, n = 4, length = 200, seed = "Emma Woodhouse")
```

```
   Emma Woodhouse!”—Emma, as love,            Kitty, only this person no infering ever, while, and
   tried very were no do be very friendly and into aid,    Man's me to loudness of Harriet's. Harriet
   belonger opinion an
```

```
markov_text(s, n = 10, length = 200, seed = "Emma Woodhouse")
```

```
   Emma Woodhouse's party could be acceptable to them, that if she ever were disposed to think of
   nothing but good. It will be an excellent charade remains, fit for any acquainted with the child
   was given up to them.
```

Notes and Hints

Hint: A good function for performing the random choice is the `choices()` function in the `random` module. You can use it like this:

```
import random

options = ["One", "Two", "Three"]
weights = [1, 2, 3] # "Two" is twice as likely as "One", "Three" three times as likely.

random.choices(options, weights)

   ['One'] # output
```

The first and second arguments must be lists of equal length. Note also that the return value is a list -- if you want the value *in* the list, you need to get it out via indexing.

Hint: The first thing your function should do is call `count_ngrams` above to generate the required dictionary. Then, handle the logic described above in the main loop.

In [4]:
```
# write markov_text() here

import random

def markov_text(s,n,length,seed):
    """
    A function that generates synthetic text according to an nth-order Markov model using s, the string, and
    adding the text to the provided seed. Generates text until it is the length specified.

    parameter 1: s, the input string of real text
    parameter 2: n, the order of the model
    parameter 3: length, the size of the text to generate, default value is 100
    parameter 4: seed, the initial string that gets the Markov model started; any subset of s of at least
    length n+1 will work

    returns text_so_far, which is the seed plus the new added text till the length is reached
    """
    D = count_ngrams(s,n+1)                    # using count_ngrams to create a dict of n+1 grams sequences
    text_so_far = seed                         # adding seed to a new variable text_so_far, that we can add new
                                               # characters to

    for i in range(length):                    # continue this until the length given is reached

        subDict = {}                           # creating an empty subDict, where we'll add all the matches found

        for key in D:                          # looping through each key in D
            if text_so_far[-n:]==key[:n]:      # if the last n letters of our text match n letters in a key
```

```
                subDict[key] = D[key]           # add that key and its corresponding value to subDict

        options = list(subDict.keys())          # add n-grams sequences from subDict to a list, options
        weights = list(subDict.values())        # add the sequences' frequencies/subDict values to a list, weights

        x = random.choices(options,weights)[0]  # picking a random option, or key


        text_so_far += x[-1]                    # add the last character of the chosen key to text_so_far


    return text_so_far
```

In [5]:
```
# try out your function for a few different values of n
print(markov_text(s, n = 2, length = 200, seed = "Emma Woodhouse"))
```

```
Emma Woodhouse's succam com Misfining, is all spery of. Marn fing ty to sit shome—aft posinuen annor do fin annot u
per, If it wo, a le, my thou burat do ther, Mrs mis mour of his any afted a hat him, a wrobselly, a
```

In [6]:
```
# try another value of n!
print(markov_text(s, n = 4, length = 200, seed = "Emma Woodhouse"))
```

```
Emma Woodhouse or a moded he small not tribute into Miss Bates, it wish you mightley that her rule, gration; “any t
hem; but if the day's boaste for ever belief from the must thout paper whom well. He was now her de
```

In [7]:
```
# try third value!
print(markov_text(s, n = 10, length = 200, seed = "Emma Woodhouse"))
```

```
Emma Woodhouse what she liked—but she could not have visited me!” she cried, looking at the book open on the table
this morning?”“Almost every moment. She has been here a whole year, and has fitted up his house, an
```

Exercise 4

Using a `for` -loop, print the output of your function for `n` ranging from `1` to `10`.

Then, write down a few observations. How does the generated text depend on `n`? How does the time required to generate the text depend on `n`? Do your best to explain each observation.

In [8]:
```
# code here
import time

for n in range(1,11):                       # running the markov_text function 10 different times, once for each
                                            # value of n from 1 and up to but not including 11
```

```
    start = time.time()                     # start time of function for n
    print(markov_text(s, n, length = 200, seed = "Emma Woodhouse")) # printing the results
    end = time.time()                       # end time of function for n
    print("The time it takes to run the function when n =",n, "is", (end-start)) # print elapsed time for function
                                                                                  # running for n
    print('\n')                             # add new line after every time the function is run for easy
                                            # readability
```

Emma Woodhoused m blede?” toonk, tedanwn mpre I buchese Mrermat owizangis at sor ry. llltild testto thandderthilesh e. is owh anke, sur k so timitofas ofavequptorr. mmeg owar are lf wonouthabule Heane atrdisulknd,
The time it takes to run the function when n = 1 is 0.08132100105285645

Emma Woodhousely. Mis mare for fardes forthersusty. Gradvily ind min gooks: be a not but on ma. Octed,” stereen beg urejoys grefor of the felf!—ands ce!—ifer and say, feedis quits, at wher, most sherit?”“My of lay s
The time it takes to run the function when n = 2 is 0.23225712776184082

Emma Woodhousefuse, this be alar a ver—Oh! Mr. Knight fore to be a pare delighter not with that her objectice. It w as very would no drawn her at any withould not of man, anding will been not victingly—'Pray, an end
The time it takes to run the function when n = 3 is 0.5995359420776367

Emma Woodhouse his feeliness.”“I thing a greater hour picture! They well you are brough I thing, her again perhaps you.”Emma think or a few day. Mrs. Goddard of it. It was and altogether belong what I though poor c
The time it takes to run the function when n = 4 is 1.2064497470855713

Emma Woodhouse's feelings, the eldest feel. Emma; “to me to her faces county, throw in general, thought think him a nd taste; and sought have her of the promoted to be an opposing—so pretty good-will. I cannot be ve
The time it takes to run the function when n = 5 is 1.9397361278533936

Emma Woodhouse an old maid who will not be the wretch.”“Pray, Mr. Knightley. We invalids think I am not to herself; these word for a truer friend and she is detestablished to go through every thing.Her first displa
The time it takes to run the function when n = 6 is 2.7792880535125732

Emma Woodhouse was so clever at a common sense. She is so with books? And I have a thorough sweet verses and niece s!—I shall not you were in the case at presently, “very much superadded decision of attend a sick ch
The time it takes to run the function when n = 7 is 3.4592981338500977

Emma Woodhouse grateful little difficulty in comprehending. Read it in comfortable with the housekeeper in incessan t conversation, recommendation, had never met with a sigh. “I am afraid of unwholesome. Serle under
The time it takes to run the function when n = 8 is 3.9197521209716797

Emma Woodhouse, what can we do about the country, that the intellects of Highbury gossips!—Tiresome wretch.”“Pray, Mr. Knightley, to be depended on; and Isabella, the usual doer of all commission, what infinitely s
The time it takes to run the function when n = 9 is 4.392858028411865

```
Emma Woodhouse's account.” CHAPTER V “I do not pretend to say that I might not see one in a hundred with gentlemen,
she thought so much of her companionableness: but dear Emma was immediately afterwards it was, Mr.
The time it takes to run the function when n = 10 is 4.8134801387786865
```

[Discuss here]

1) The higher n is, the more real-sounding the sentences are. In other words, the words that the characters that are added onto the seed form are more real rather than gibberish like in the lower n examples. This is because with a higher n, there are more likely to be real proper word combinations that makes sense. For example, if n is just 2, and you are looking for l, you will have a lot of n-gram combinations like li, le, lo, la, lu. With a higher n, you will find more word combinations that make more sense.

2) If n is higher, the time required to generate the text is higher also, as shown by our output. This is because there are more keys when n = 10 versus when n = 2. Therefore, it takes more time to look through all of the keys and find a matching one. For a lower n, there would be less keys but higher values.